

为了使在实验板上演示实验成为可能，LS-CPU-EXB-002 试验箱实现了 LCD 触摸屏的硬件驱动，使得不需要处理器核就能使用触摸屏的显示和输入功能，且设计了简单清晰的调用接口方便学生使用。

为降低学生在 FPGA 硬件平台上的实际操作难度，方便教师指导，特针对组成原理和体系结构课程编写了本实验指导手册。指导手册主要分为两部分，cpu 设计部分和 cpu 优化部分：前者主要为组成原理需要，包含实验一到实验七，以及课程设计；后者主要为体系结构需要，包含课程设计和课程设计拓展。

具体实验内容如下：

实验一 数据运算：定点加法(软硬件平台入门)

实验二 数据运算：定点乘法

实验三 寄存器堆实现

实验四 ALU 模块实现

实验五 存储器实现

实验六 单周期 CPU 实现

实验七 多周期 CPU 实现

课程设计 静态 5 级流水线 CPU 实现

课程设计拓展 优化 CPU 系统

二、实验一 数据运算：定点加法

1 实验目的

1. 熟悉 LS-CPU-EXB-002 实验箱和软件平台。
2. 掌握利用该实验箱各项功能开发组成原理和体系结构实验的方法。
3. 理解并掌握加法器的原理和设计。
4. 熟悉并运用 verilog 语言进行电路设计。
5. 为后续设计 CPU 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 阅读 LS-CPU-EXB-002 实验箱相关文档，熟悉硬件平台，特别需要掌握利用显示屏观察特定信号的方法。学习软件平台和设计流程。
2. 熟悉计算机中加法器的原理。
3. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，本次实验的加法器可以使用全加器自己搭建加法模块，也可以在 verilog 中直接使用“+”（系统是自动调用库里加法 IP，且面积时序更优），依据教师要求选择一种方法实现。
4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 2.1。外围模块中需调用封装好的触摸屏模块，显示两个加数和加法结果，且需要利用触摸功能输入两个加数。

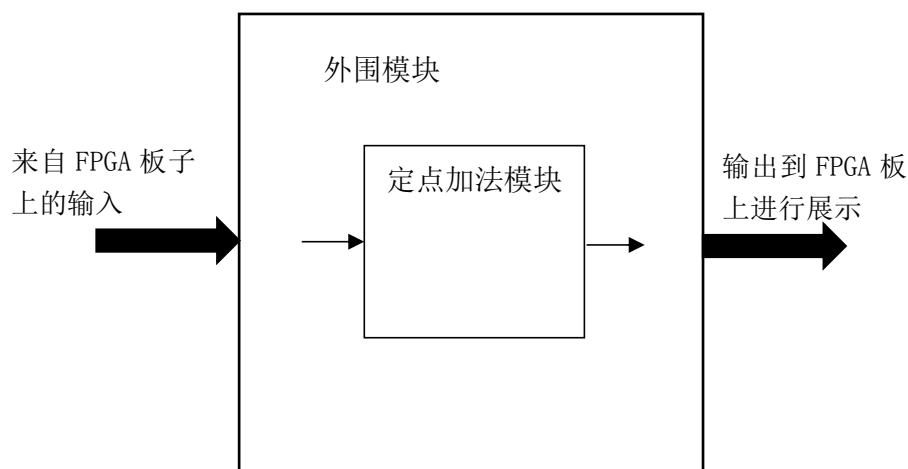


图 2.1 定点加法设计实验的顶层模块大致框图

7. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板上进行演示。

4 实验要求

1. 做好预习：

- 1) 了解软硬件平台；
- 2) 掌握定点加法的工作原理；
- 3) 确定定点加法的输入输出端口设计；
- 4) 在课前画好设计框图或实验原理图；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 2.1。

2. 实验实施：

- 1) 确认定点加法的设计框图的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用定点加法模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考实现

本部分给出了定点加法实验的参考设计方案，详细阐述了软硬件平台的使用方法，特别是 FPGA 实验板上的 LCD 触摸屏的调用模块的使用。

5.1 实验步骤

1) 新建工程

启动 vivado 软件，在菜单栏点击“File”->“New Project”，出现新建工程向导，选择“Next”，输入工程名称，选择工程的文件位置，然后选择“Next”：

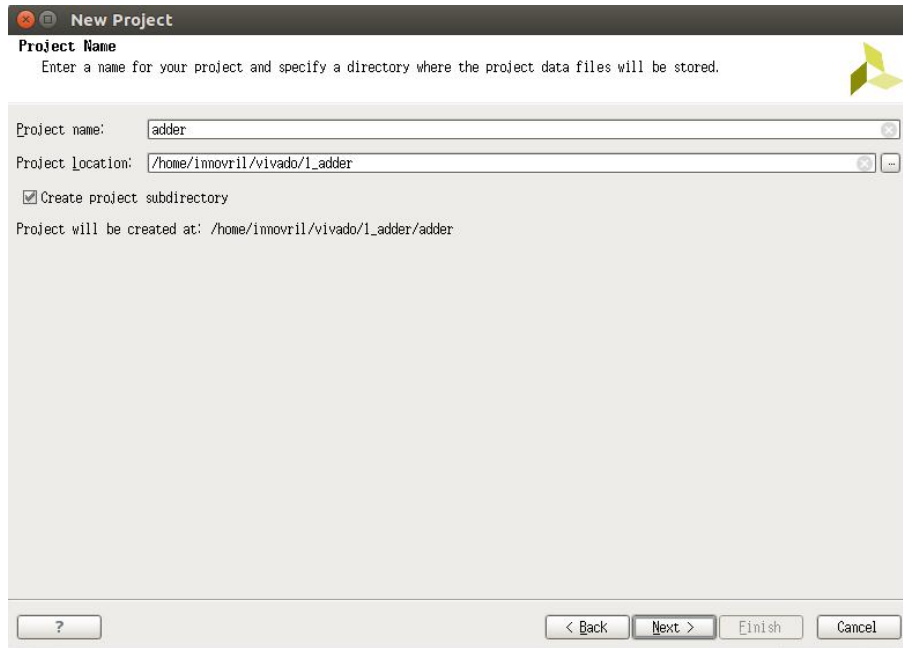


图 2.2 设置工程名称和位置

选择”RTL Project”，勾选”Do not specify sources at this time”，点击”Next”：

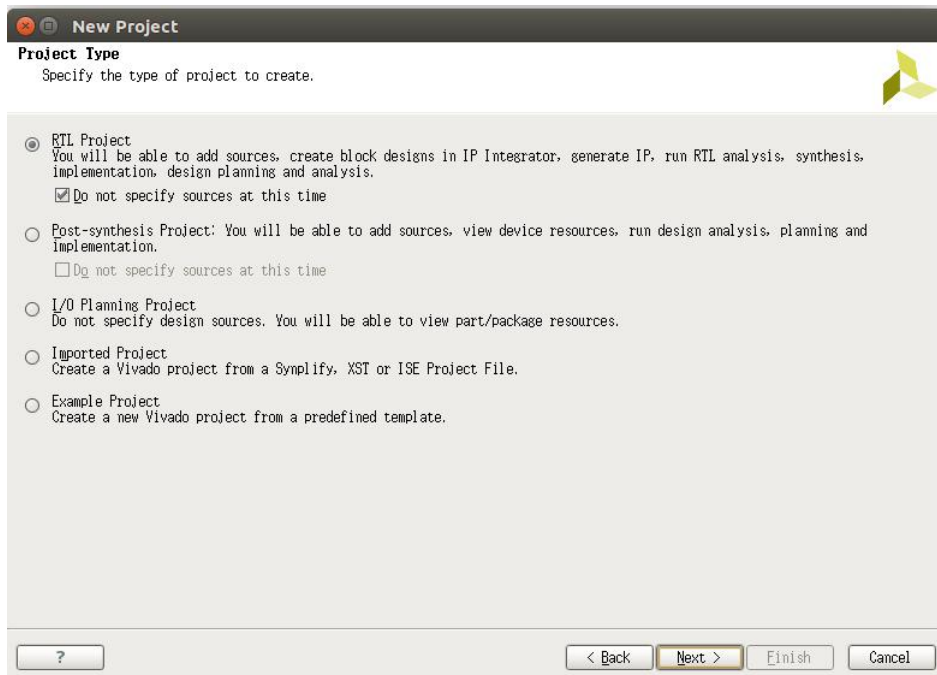


图 2.3 工程设置

在筛选器的”family”选择”Artix 7”，”package”选择”fbg676”，在筛选得到的型号里面选择”xc7a200tfbg676-2”：

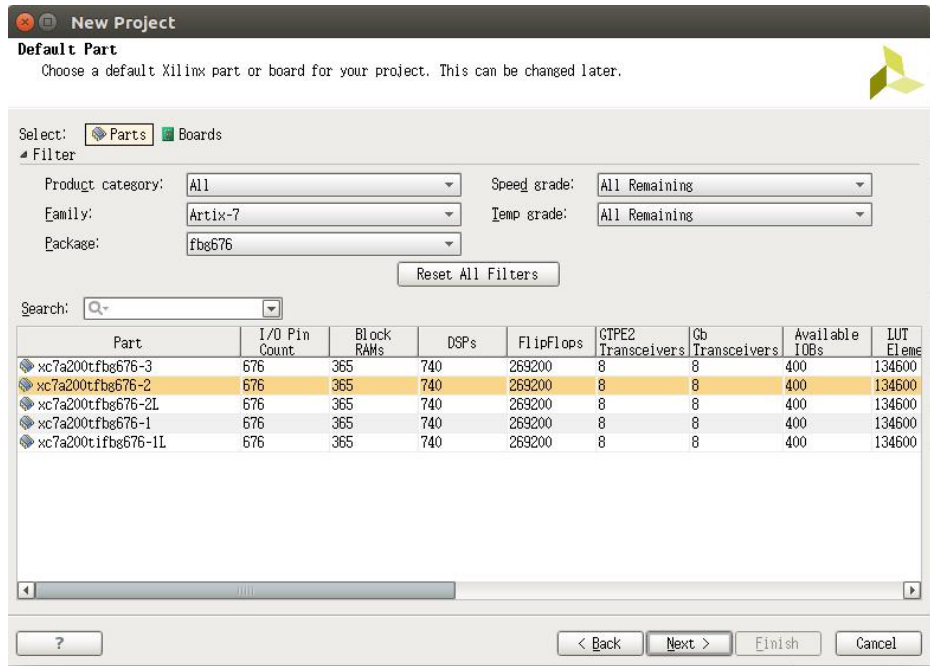


图 2.3 工程设置

然后选择“Next”，出现下图界面，点击“Finish”。

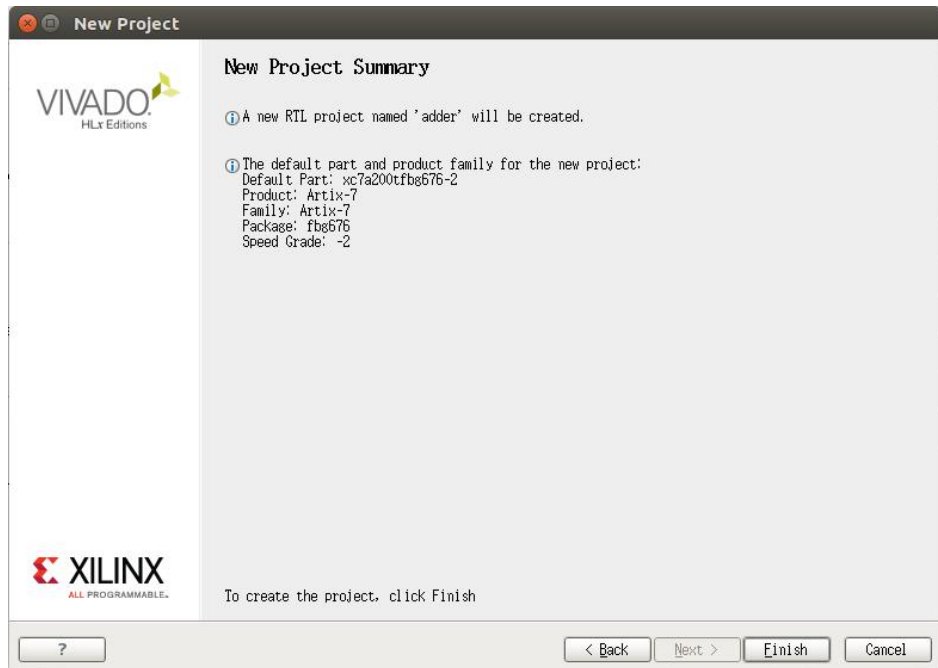


图 2.4 工程信息

2) 添加源文件

verilog 代码都是以“.v”为后缀名的文件，可以在其他文件编辑器里写好，再添加到新建的工程中，也可以在工程中新建一个再编辑。

添加已有 verilog 文件的方法如下：在”Project Manager”下点击”Add sources”，选择”Add or create design sources”



图 2.5 添加源文件

点击”Next”，可以看到如下的界面，可以按文件添加或者按文件夹添加，也可以自己创建：

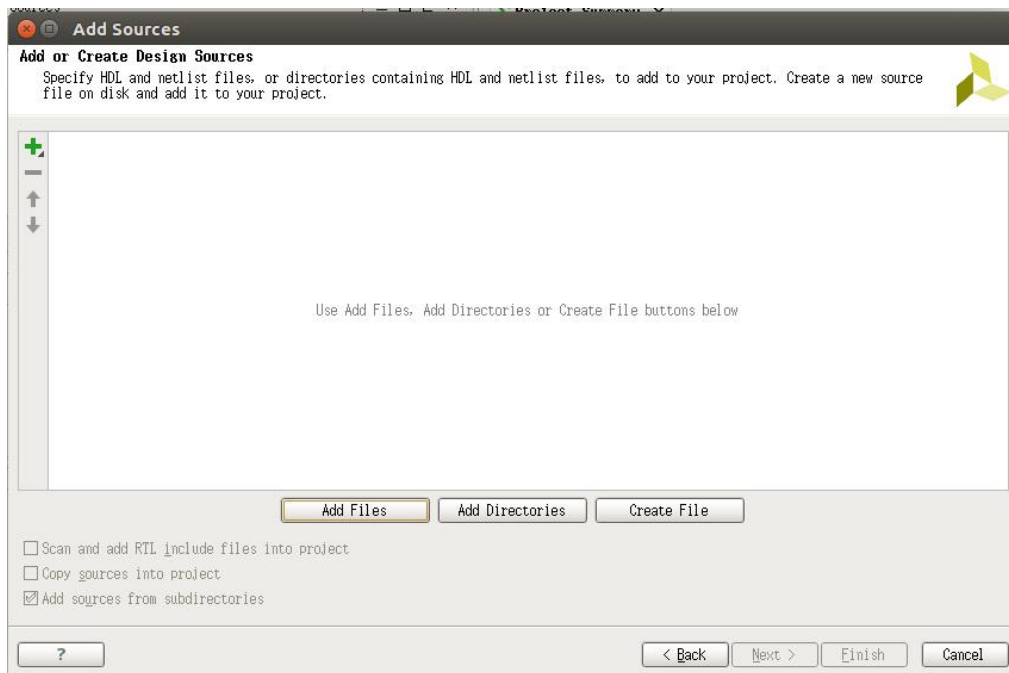


图 2.6 添加源文件

点击”Add Files”，选择 adder.v，点击”OK”：

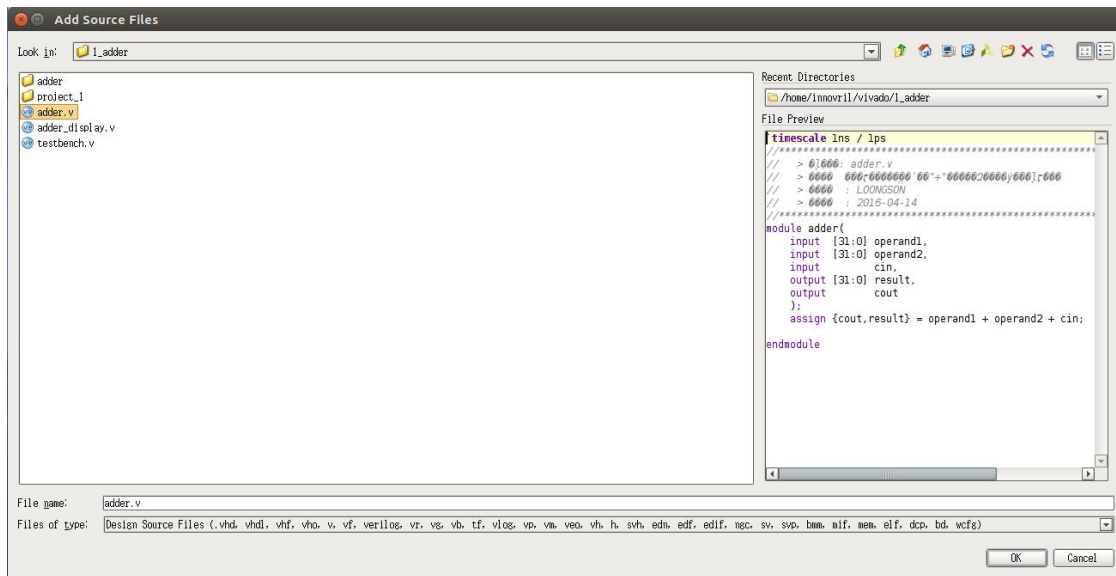


图 2.6 添加源文件

adder.v 的代码如下：

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名： adder.v
4  //    > 描述   ： 加法器，直接使用"+", 会自动调用库里的加法器
5  //    > 作者   ： LOONGSON
6  //    > 日期   ： 2016-04-14
7  //*****
8  module adder(
9      input  [31:0] operand1,
10     input  [31:0] operand2,
11     input                cin,
12     output [31:0] result,
13     output                cout
14 );
15     assign {cout,result} = operand1 + operand2 + cin;
16
17 endmodule

```

代码比较简单，有 2 个 32 位数的输入和 1 个进位输入，产生 1 个 32 位的加法和结果和 1 个向高位的进位。本实验提供的参考设计是直接写“+”号实现加法功能的，这样的写法综合工具会调用内部的模块库的加法器来实现，往往会比自行设计的加法模块更高效和省资源。

本例中，adder.v 为定点加法实验的主体代码，由于实验较简单，故只有这一个.v 文件，对于以后的 CPU 实验，则会有多个.v 文件，形成一定的调用层次。

3) 添加展示外围模块

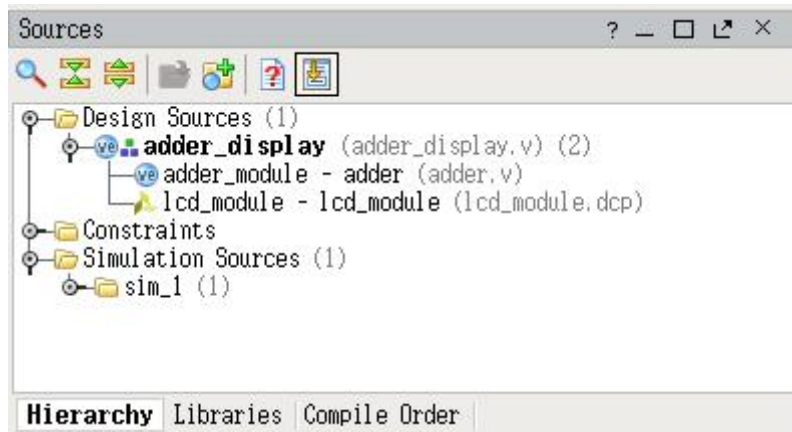


图 2.9 添加 lcd_module 文件成功

至此，代码实现都已经完成，下面需要对代码功能进行仿真验证功能的正确性。

4) 功能仿真

在进行功能仿真时，需要先建立一个 testbench(测试平台)。一个比较完备的 testbench 就是产生输入激励，送入到要测试的功能模块里，然后读出功能模块的执行结果，与预期的结果进行比较，以此验证功能模块的正确性。但在目前的实验设计上，只需要一个最简单的 testbench——只产生输入激励即可。

在本例中，需要产生的输入激励就是 2 个加数和 1 个低位进位信号，在该激励输入到加法功能模块中后，会输出加法结果和向高位的进位信号。仿真的过程中会产生波形文件，可以通过观察波形文件确定功能的正确性，在出错的情况下可以定位错误位置。

点击“Add Source”，选择”Add or create simulation sources”，添加“testbench.v”。

testbench.v 的代码如下：

```
1  `timescale 1ns / 1ps //仿真单位时间为 1ns，精度为 1ps
2  module testbench;
3
4      // Inputs
5      reg [31:0] operand1;
6      reg [31:0] operand2;
7      reg cin;
8
9      // Outputs
10     wire [31:0] result;
11     wire cout;
12     // Instantiate the Unit Under Test (UUT)
13     adder uut (
14         .operand1(operand1),
15         .operand2(operand2),
16         .cin(cin),
17         .result(result),
18         .cout(cout)
19     );
```

```

20    initial begin
21        // Initialize Inputs
22        operand1 = 0;
23        operand2 = 0;
24        cin = 0;
25        // Wait 100 ns for global reset to finish
26        #100;
27        // Add stimulus here
28    end

29    always #10 operand1 = $random; //$random 为系统任务，产生一个随机的 32 位数

30    always #10 operand2 = $random; //#10 表示等待 10 个单位时间(10ns)，即每过 10ns，赋
    值一个随机的 32 位数
31    always #10 cin = {$random} % 2; //加了拼接符，{$random}产生一个非负数，除 2 取余
    得到 0 或 1
32    endmodule

```

添加 testbench.v 后，工程管理区如图所示，如果 testbench.v 前面没有三个方形的 top 标志，则右键点击 testbench.v，选择”Set as Top”：

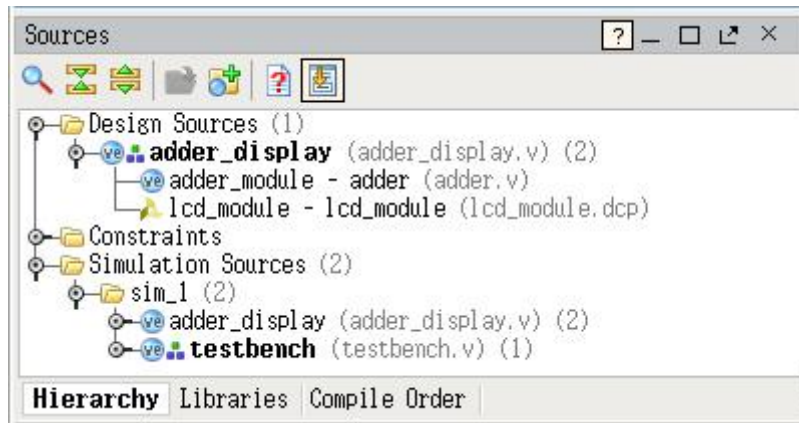


图 2.10 添加 testbench.v 成功

在左侧的导航栏中点击”Run Simulation”，选择”Run Behavioral Simulation”：

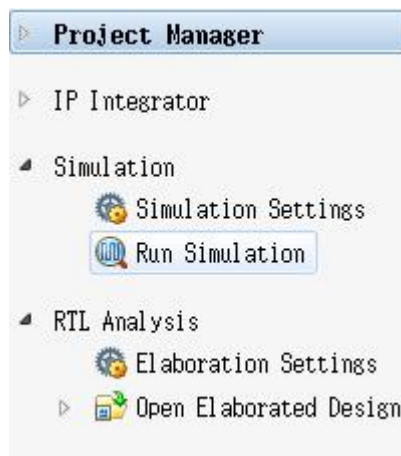


图 2.11 运行仿真

如果没有语法错误，会弹出如下的界面：

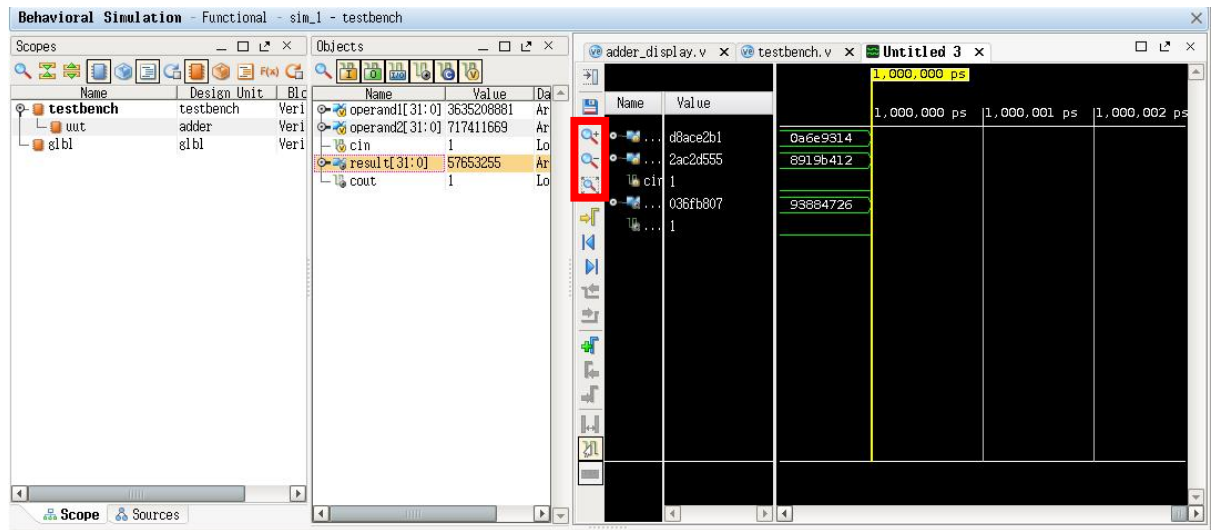


图 2.12 仿真界面

可以通过图 2.12 中红色圈出的放大、缩小、缩放全屏三个按钮以及鼠标左键点击波形界面(出现图中黄线，缩放会以黄线为中心)观察特定波形区域的信息。

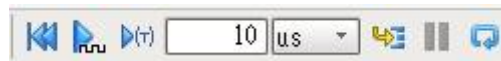


图 2.13 仿真工具栏

上图是仿真工具栏，从左到右依次是”从 0 时刻开始仿真”、”运行仿真”、”运行特定长时的仿真”、”仿真时长”、”时间单位”、”单步仿真”、”暂停”、”重新载入仿真”。

在仿真界面左侧的 “Scopes”(图 2.12 的最左列)选择要观察信号所在源文件对象，然后在“Objects”窗口(图 2.12 的第二列)可以看到该对象里所有的信号，可以左键选中要观察的信号，然后右键会出现很多可选项，选择“Add To Wave Window”添加该信号到波形窗口，如下图。

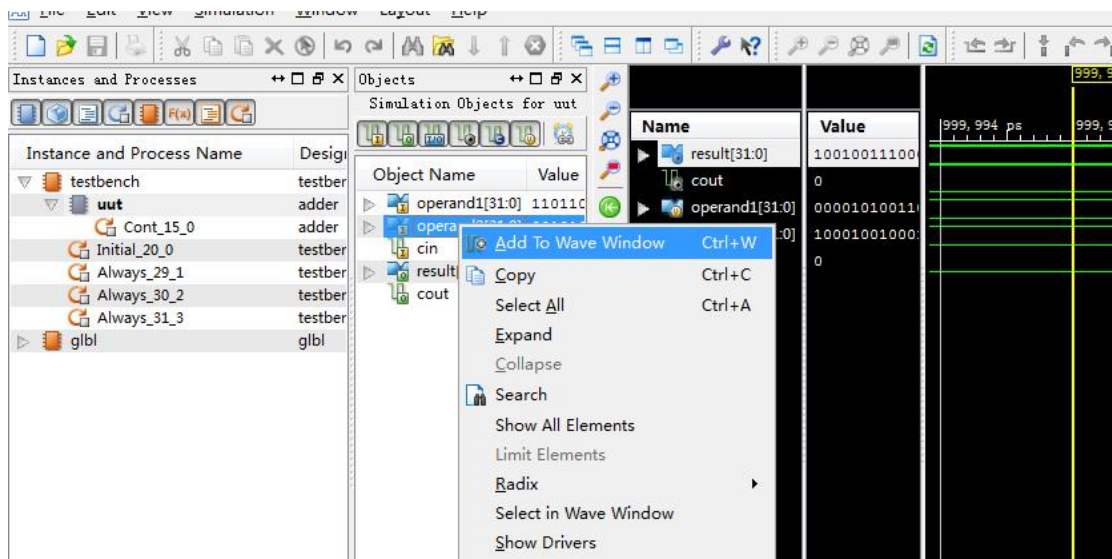


图 2.14 添加要观察的信号到波形界面

在添加新的信号到波形窗口后，需要点击 **Relaunch** 才能看到该信号的数据。由于定点加法模块内部所有信号均为输入输出信号，故此处没有新的信号可以添加。但在以后的实验中，会经常需要添加新的信号到波形窗口。

另外，当前波形窗口显示的数据均为 2 进制数，对于 32 位的数不便于观察，可以在波形窗口的 **Name** 列左键选择信号，然后右键会出现很多可选项，选择“**Radix**”可以选择要数据显示的进制。

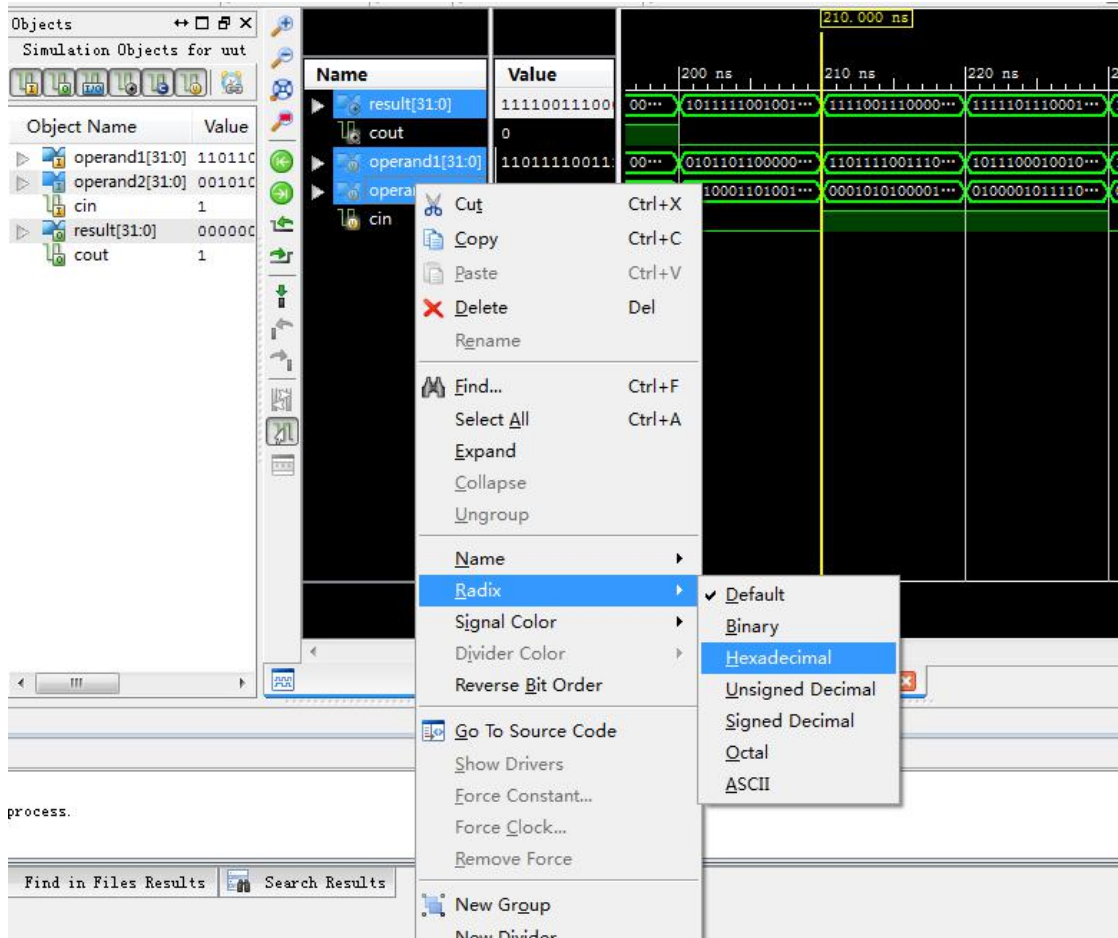


图 2.15 更改波形窗口信号的显示进制

此处选择 16 进制，结果如下图：

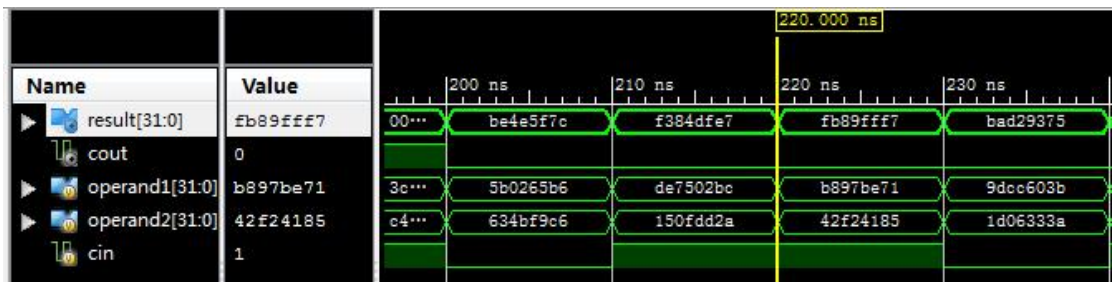


图 2.16 数据以 16 进制形式显示

可以检查几组数据，比如： $b897be71+42f24186+1=fb89fff7$ ，正确。类似的，通过观察波形可以看到加法功能模块随机测试并没有出错，我们认为功能趋于稳定，可以认为是正确的。

至此，代码编辑和功能仿真都已完成，认为功能基本正确，后续流程就是上板验证了。

5) 上板验证

所谓上板验证是指将功能代码进行综合和布局布线后下载到 FPGA 板上运行，在板上检查运行的正确性。

因此，需要设定一套在板上检查结果的机制。比如，对于本例中的定点加法，可以设定使用拨码开关作为加数的输入，使用 led 灯作为加法结果的输出，这样就能在 FPGA 实验板上观察加法模块的运行结果了。但遗憾的是，对于 32 位加法，需要 64 个拨码开关和 32 个 led 灯，但显然实验板上提供不了如此之多的 IO 接口，当然也可以通过精巧的设计使用实验板上有限的资源来完成加数输入和结果显示的，但使用起来不够方便。

对于 LS-CPU-EXB-002 试验箱并没有上述的问题，因为可以通过 LCD 触摸屏输入 32 位加数，并能显示加法结果。之前提到的外围展示模块 `adder_display.v` 就是调用 LCD 触摸屏来完成上板验证的机制的设计的，具体验证机制的解析见本章的 5.2 小节。

在有了板上验证机制后，需要添加引脚绑定的约束文件。所谓约束文件就是将顶层模块(本例中为 `adder_display`)的输入输出端口与 FPGA 板上的 IO 接口引脚绑定，以完成在板上的输入输出。

约束文件后缀名为 `.xdc`，有两种方法可以添加约束文件，其一是用“Add or create constraints”添加或创建约束文件，如下图：

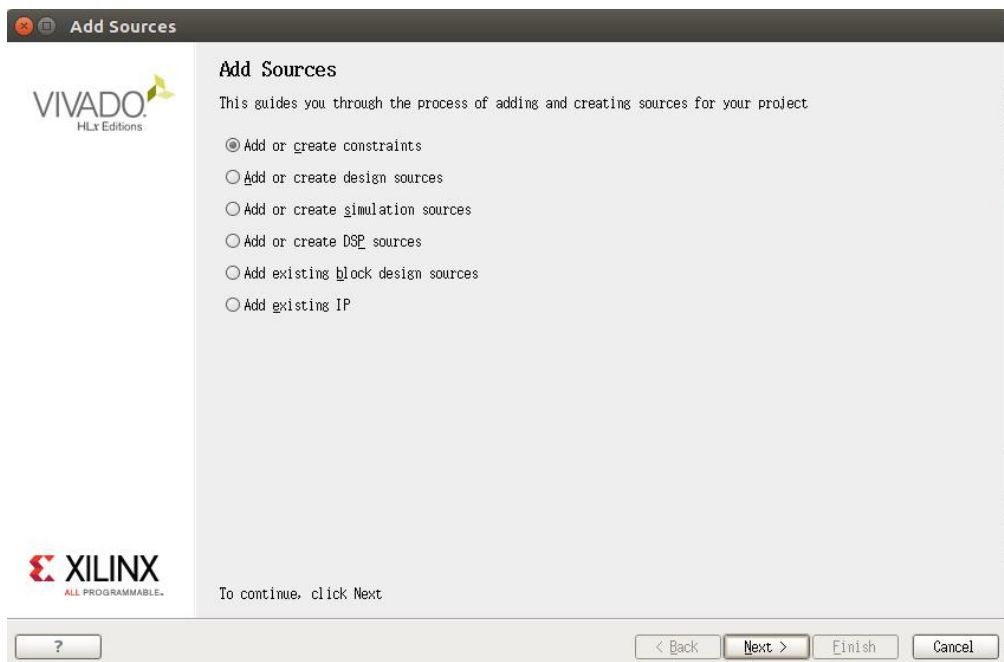


图 2.17 添加约束文件

之后选择添加 adder.xdc。

adder.xdc 内容如下：

```
1  set_property PACKAGE_PIN AC19 [get_ports clk]
2  set_property PACKAGE_PIN A3 [get_ports led_cout]
3  set_property PACKAGE_PIN Y3 [get_ports resetn]
4  set_property PACKAGE_PIN AC21 [get_ports input_sel]
5  set_property PACKAGE_PIN Y6 [get_ports sw_cin]
6
7  set_property IOSTANDARD LVCMOS33 [get_ports clk]
8  set_property IOSTANDARD LVCMOS33 [get_ports led_cout]
9  set_property IOSTANDARD LVCMOS33 [get_ports resetn]
10 set_property IOSTANDARD LVCMOS33 [get_ports input_sel]
11 set_property IOSTANDARD LVCMOS33 [get_ports sw_cin]
12
13 #lcd
14 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
15 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
16 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
17 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
18 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
19 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
20 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
21 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
22 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
23 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
24 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
25 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
26 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
27 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
28 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
29 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
30 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
31 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
32 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
33 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
34 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
35 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
36 set_property PACKAGE_PIN L19 [get_ports ct_int]
```

```
37 set_property PACKAGE_PIN J24 [get_ports ct_sda]
38 set_property PACKAGE_PIN H21 [get_ports ct_scl]
39 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
40
41 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
47 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

可以看到主要有时钟与复位信号的引脚连接，led 灯和拨码开关的引脚连接，以及 LCD 触摸屏引脚的连接。此处时钟与复位信号和 LCD 触摸屏引脚都是 lcd_module 需要用到的，可以不用管。

对于 led 灯的连接（led_cout 绑定到 A3 引脚上），可以看到是用来显示加法向高位的进位的。

对于拨码开关的连接，“sw_cin”是用来输入低位的进位的，而“input_sel”是用来选择通过触摸屏输入的 32 位数据为加数 1 还是加数 2。

由于以后的实验都需要用到 LCD 触摸屏，故 LCD 触摸屏相关引脚的绑定是固定不变的，故建议以后实验都通过添加已有 xdc 文件，然后再根据需求修改 led 和拨码开关等引脚的绑定。

也可以通过 Vivado 中的 I/O Planning 功能来产生约束文件，具体过程如下：

点击 Flow Navigator 中 Synthesis 下 Run Synthesis，进行综合：

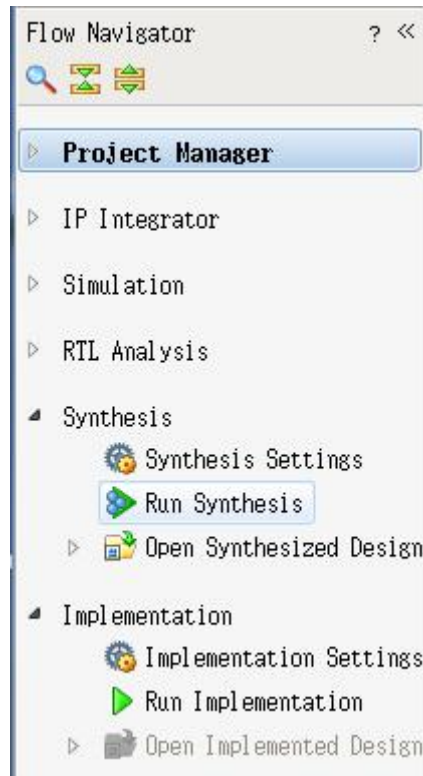


图 2.18 运行综合

综合完成后，选择“Open Synthesized Design”，查看综合结果：

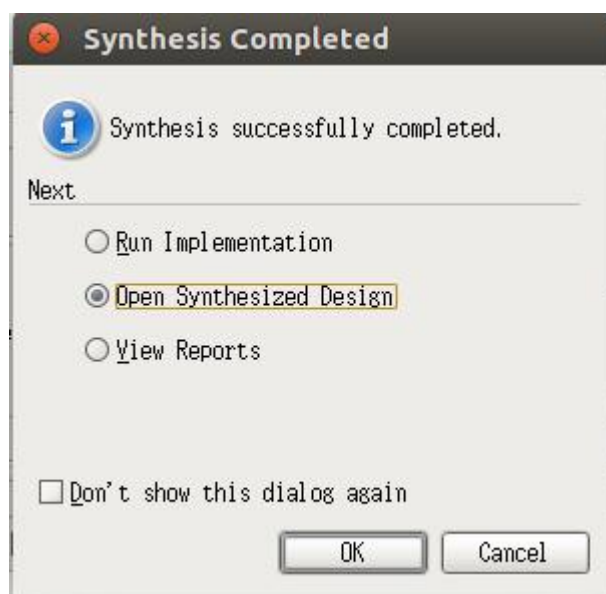


图 2.19 综合完成

得到如下界面：

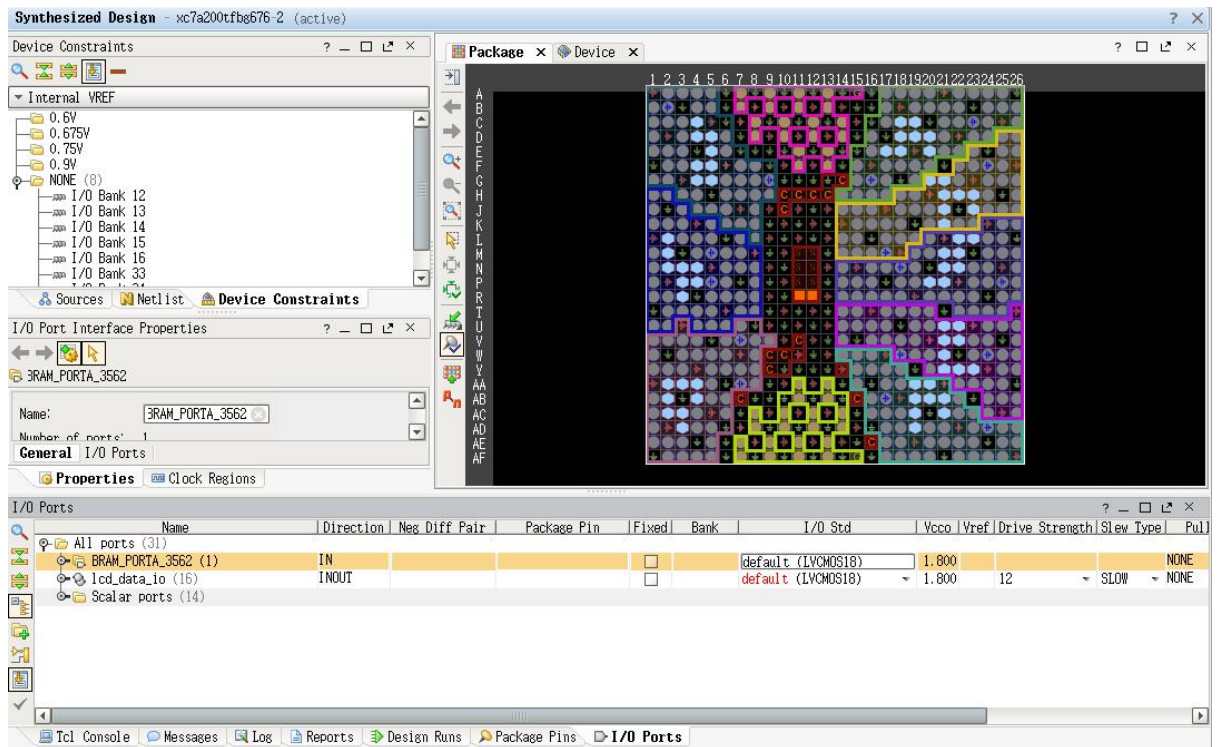


图 2.20 综合结果显示

之后参照实验板原理图和引脚对应关系表，在 I/O Ports 栏下填入正确的 Package Pin，针对本实验板，I/O Std 要统一设置为 LVCMOS33。

所有约束条件填写完成之后效果如下：

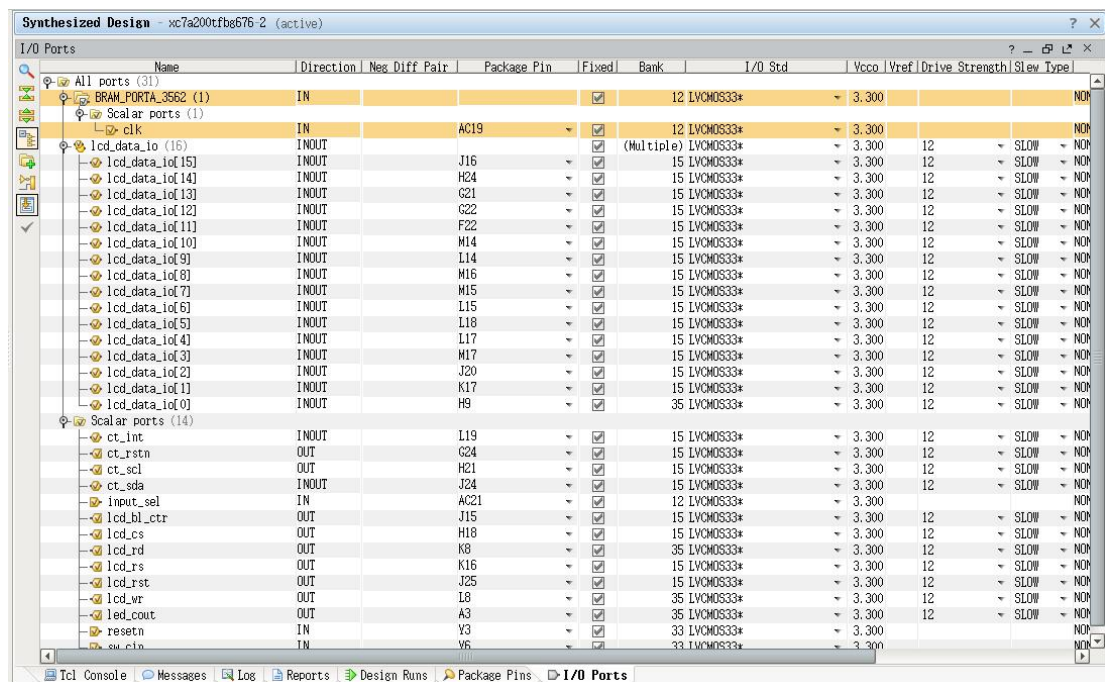


图 2.21 填写完成的 I/O Planning

点击右上角的“保存”键即可保存为 xdc 文件。

后续的流程就是综合、布局布线和产生可烧写文件，可以依次双击运行，也可以只双击“Generate Bitstream”会自动运行这三步，运行结果如下，可以选择 Open Implemented Design 来查看实现结果：

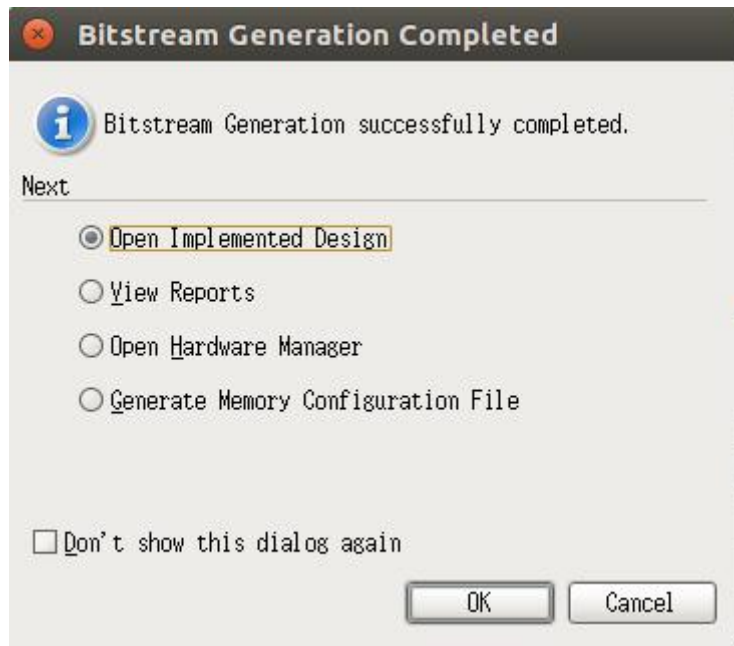


图 2.22 综合、布局布线和产生可烧写文件运行结果

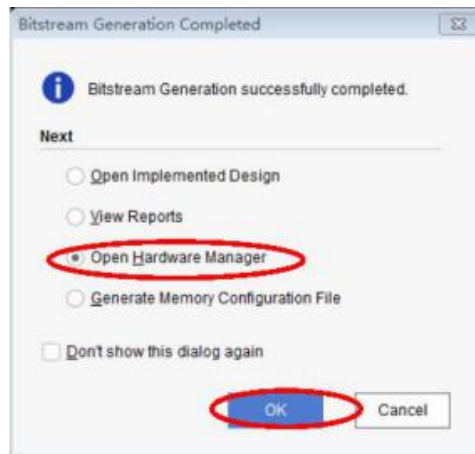
这时可烧写的文件已经产生成功了，后缀名为.bit。

在打开 FPGA 实验板，上电，并将下载线与电脑相连后，打开电源，FPGA 板如下图所示：



图 2.23 上电后初始的 FPGA 板

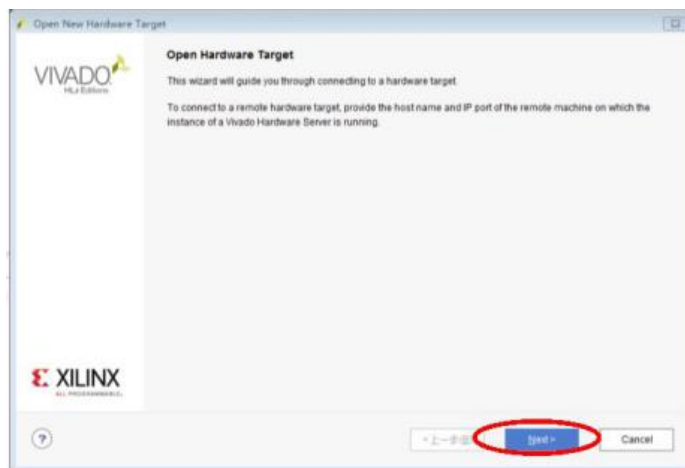
在比特流文件生成完成的窗口选择“Open Hardware Manager”，进入硬件管理界面。连接 FPGA 开发板的电源线与与电脑的下载线，打开 FPGA 电源。如下图：



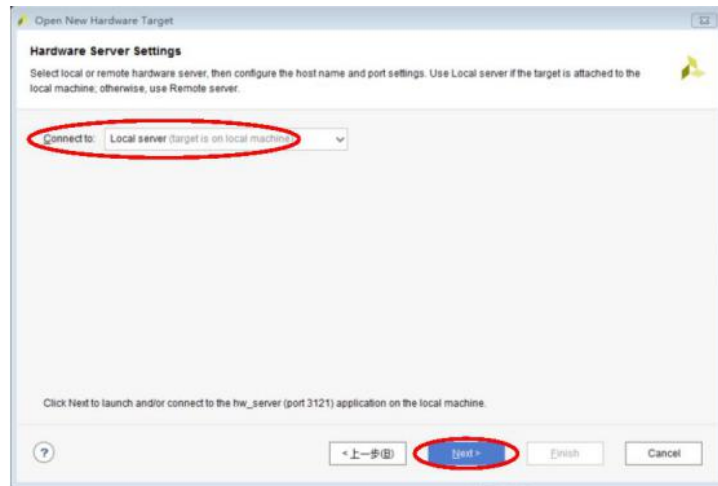
在“Hardware Manager”窗口的提示信息中，点击“Open target”的下拉菜单的“Open New Target”（或在“Flow Navigator”下“Program and Debug”中展开“Open Hardware Manager”，点击“Open Target”->“Open NewTarget”）。也可以选择“Auto Connect”自动连接器件。



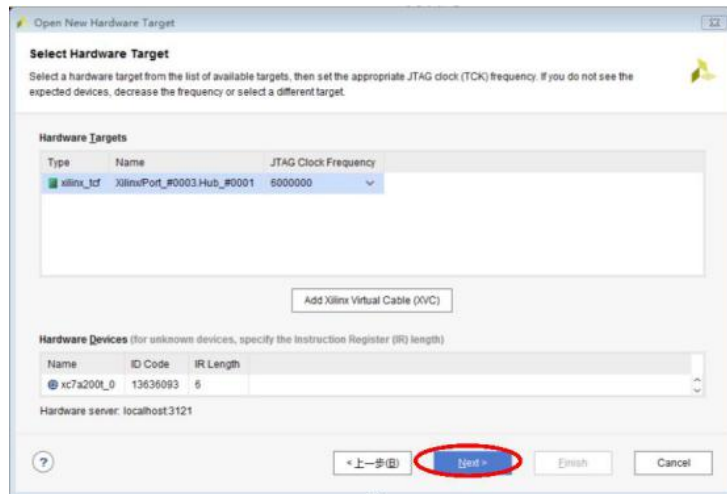
在“Open Hardware Target”向导中，先点击“Next”，进入 Server 选择向导。如下图：



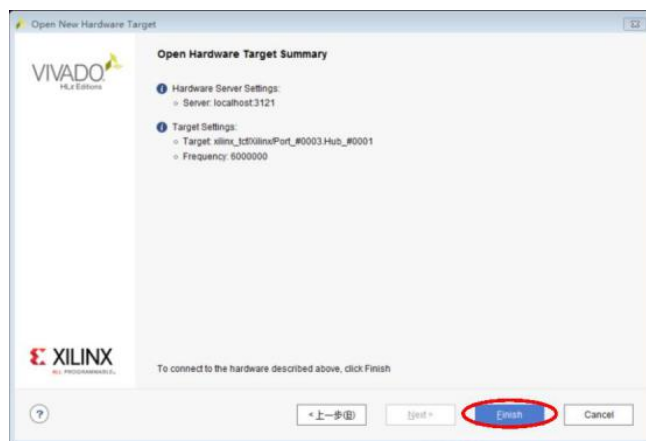
选择连接到“Local server”，点击“Next”。出现以下图：



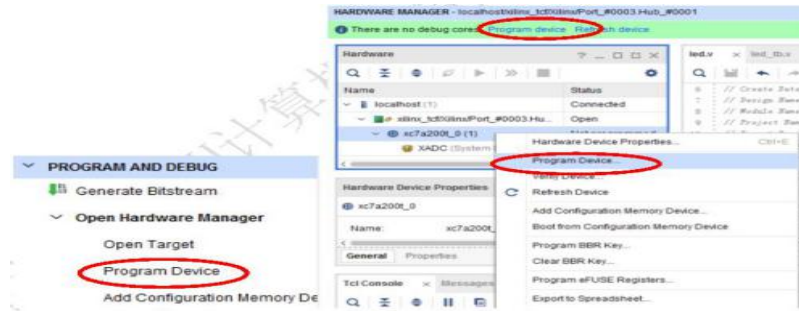
选择目标硬件，点击“Next”。



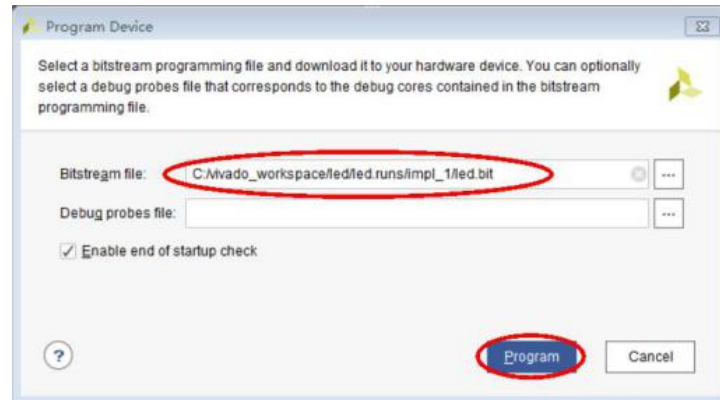
完成目标硬件打开。点击“Finish”。



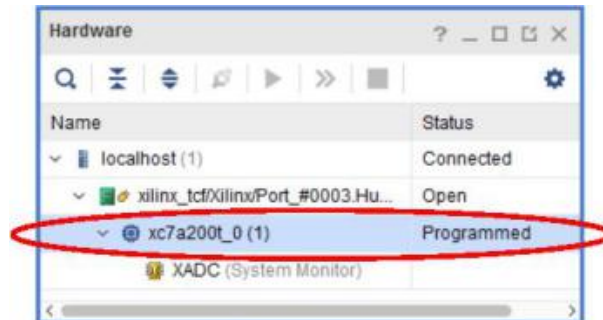
对目标硬件编程。在“Hardware”窗口右键单击目标器件“xc7a200t_0”，选择“Program Device...”。或者“Flow Navigator”窗口中“Program and Debug”->“Hardware Manager”->“Program Device”。



选择下载的 bit 流文件，点击“Program”。



完成下载后，“Hardware”窗口下的“xc7a200t_0”的状态变成“Programmed”。



烧写 bit 文件，烧写完成后的 FPGA 板显示如下：



图 2.24 加法模块下载成功后的 FPGA 板

从图 2.24 中可以看到 LCD 触摸屏上分别显示了 2 个加数和加法结果，最右侧的 led 灯为向高位的进位，该 led 灯为共阴极的，即输入为 0 是亮，为 1 是不亮，由于当前进位为 0，故 led 灯为亮。拨码开关最右侧的开关用来选择触摸屏输入的数据为加数 1 还是加数 2，当前该开关置为 1，说明触摸屏输入的 32 位数位加数 2。

在需要使用触摸屏的输入功能时，手指点击屏上最后一列的“START INPUTING”即可进入输入模式，如下图：



图 2.25 LCD 屏进入输入模式

从图 2.25 中可看到，触摸屏的输入模式为小键盘，包含 0~F，回退键(BACK)和确认键(OK)。输入的数据为 32 位的 16 进制数，当输出错误的时候，可以按 BACK 键回退一格，当输入完成时按 OK 键完成输入，同时会退出输入模式。当按 OK 时，输入未满 32 位，则会高位补 0，比如：只输入了 123，则最终屏调用模块输入时，输入数据为 0x00000123。

在本例中，给加数 2 输入 0x11111111 后，可看到 FPGA 板上结果如下图：



图 2.26 加数 2 输入 0x11111111

可看到加法结果页变为 0x11111111，正确。可以将拨码开关最右侧的置为 0，此时表示输入数据为加数 1。因此，通过触摸屏可以给加数 1 和加数 2 输入任意 32 位数，并能实时看到加法结果，完成上板验证。

5.2 LCD 触摸屏调用方法

正如前面所述，`adder_display.v` 作为顶层模块，里面实例化了 `adder.v`，同时也调用了 `lcd_module` 用来上板演示。此小节着重讲解 `adder_display.v` 里调用 LCD 触摸屏的方法。

1) LCD 触摸屏调用接口

先来看 `lcd_module.v` 的代码：

```
1  //*****
2  //    > 文件名: lcd_module.v
3  //    > 描述   : lcd 触摸屏模块, 为黑盒文件
4  //    > 作者   : LOONGSON
5  //    > 日期   : 2016-04-14
6  //*****
7  //synthesis attribute box_type <lcd_module> "black_box"
8  module lcd_module(
9      input  clk,          //10Mhz
10     input  resetn,       //低使能
11
12     //调用触摸屏的接口
13     input      display_valid,
14     input  [39:0] display_name,
15     input  [31:0] display_value,
16     output [ 5:0] display_number,
17     output      input_valid,
18     output [31:0] input_value,
19
20     //lcd 触摸屏相关接口, 不需要更改
21     output reg   lcd_rst,
22     output       lcd_cs,
23     output       lcd_rs,
24     output       lcd_wr,
25     output       lcd_rd,
26     inout [15:0] lcd_data_io,
27     output       lcd_bl_ctr,
28     inout        ct_int,
29     inout        ct_sda,
30     output       ct_scl,
31     output       ct_rstn
32 );
33 endmodule
```

可以看到该模块为黑盒文件, 其中时钟和复位信号以及 LCD 触摸屏引脚接口不需要关注, 下面介绍调用触摸屏的 6 个接口。

2) 调用 LCD 触摸屏显示

```
input      display_valid,
input  [39:0] display_name,
input  [31:0] display_value,
output [ 5:0] display_number,
```

display 的 4 个接口用于在屏上显示数据。从图 2.24 FPGA 板的实物图中可以看到, LCD 屏用于显示的区域块共有 2 列, 22 行, 故共可显示 44 组数据。

display_number 就是输出到外部说明当前需要显示的区域块为第几块, 有效编号从 1~44, 指示 44 块显示区域块。

每块显示区域块可显示 14 个字符，其中头五个字符为块名，指示当前块显示的数据的意义，由 `display_name` 输入指定。`display_name` 输入的为要显示字符的 ASCII 码，5 个 ASCII 码共 40 位。块名可显示字符为大写的 26 个字母，下划线“_”，0~9 数字以及空格。

每块显示区域块的第 6 个字符为冒号，用于区分块名和块数据段。后 8 个字符显示该块的数值，显示的为 32 位 16 进制数，故占用 8 个字符。该段由 `display_value` 输入指定，`display_value` 输入的为 32 位 2 进制数，`lcd_module` 内部会自动转换为 8 个字符显示。

最后还有 `display_valid` 输入，用于指示是否需要在当前显示区域块(由 `display_number`)显示数据，为 1 有效。

在本例中，顶层模块 `adder_display.v` 调用 LCD 触摸屏显示功能的代码如下：

```
117 //-----{输出到触摸屏显示}begin
118 //根据需要显示的数修改此小节，
119 //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
120 //44 块显示区域从 1 开始编号，编号为 1~44，
121     always @(posedge clk)
122     begin
123         case(display_number)
124             6'd1 :
125                 begin
126                     display_valid <= 1'b1;
127                     display_name  <= "ADD_1";
128                     display_value <= adder_operand1;
129                 end
130             6'd2 :
131                 begin
132                     display_valid <= 1'b1;
133                     display_name  <= "ADD_2";
134                     display_value <= adder_operand2;
135                 end
136             6'd3 :
137                 begin
138                     display_valid <= 1'b1;
139                     display_name  <= "RESUL";
140                     display_value <= adder_result;
141                 end
142             default :
143                 begin
144                     display_valid <= 1'b0;
145                     display_name  <= 40'd0;
146                     display_value <= 32'd0;
147                 end
148         end
149     end
```

```

148         endcase
149     end
150 //-----{输出到触摸屏显示}end
151 //-----{调用触摸屏模块}end-----//

```

可以看到加法模块只需要区域块 1~3 用于显示，其他块(default 分支)的 display_valid 为 0。另外，display_name 接口赋值 5 位字符组成的字符串即可，字符串里应当只出现大写的 26 个字母，下划线“_”，0~9 数字或空格，其他字符在 LCD 触摸屏上暂不支持显示。而 display_value 直接赋值 32 位的数据即可。

3) 调用 LCD 触摸屏输入

```

output        input_valid,
output [31:0] input_value,

```

input 的两个接口用于使用触摸屏的输入功能。从图 2.24 可以看到，当需要使用输入功能时，触摸屏底部的“START INPUTING”栏即可进入输入模式，见图 2.25。点击屏小键盘上的 OK 键完成输入，会退出输入模式，同时 lcd_module 会拉高 input_valid 信号一拍，表示有数据要输出，而输出数据 input_value 会依据之前的输入确定，当输入不足 32 位时，会自动高位补 0，比如：只输入了 123，就按 OK 键，则最终 input_valid 的值为 0x00000123。当输入有误时，可以按 BACK 键回退一格。

在本例中，顶层模块 adder_display.v 调用 LCD 触摸屏输入功能的代码如下：

```

87 //-----{从触摸屏获取输入}begin
88 //根据实际需要输入的数修改此小节，
89 //建议对每一个数的输入，编写单独一个 always 块
90     //当 input_sel 为 0 时，表示输入数为加数 1，即 operand1
91     always @(posedge clk)
92     begin
93         if (!resetn)
94             begin
95                 adder_operand1 <= 32'd0;
96             end
97         else if (input_valid && !input_sel)
98             begin
99                 adder_operand1 <= input_value;
100             end
101     end
102
103     //当 input_sel 为 1 时，表示输入数为加数 2，即 operand2
104     always @(posedge clk)
105     begin
106         if (!resetn)
107             begin
108                 adder_operand2 <= 32'd0;
109             end
110         else if (input_valid && input_sel)

```

```
111         begin
112             adder_operand2 <= input_value;
113         end
114     end
115 //-----{从触摸屏获取输入}end
```

可以看到加法模块需要使用触摸屏输入 2 个加数，故需要一个拨码开关指示输入的数据为加数 1 还是加数 2。故理论上，触摸屏加上外部的选择信号，可以给任意信号输入 32 位的数，比如可以输入一条 32 位的指令，可以输入内存的 32 位地址等等，这些在后续的 CPU 实验中可能会经常用到。当需要使用触摸屏输入多个数据时，比如本例中输入 2 个加数，建议每个数据单独用一个 always 块采集输入，这样的写法更接近电路实际情况，示例中的代码就是如此写的。

4) adder_display.v 完整代码

```
//*****
1  *
2  //    > 文件名: adder_display.v
3  //    > 描述   : 加法器显示模块，调用 FPGA 板上的 IO 接口和触摸屏
4  //    > 作者   : LOONGSON
5  //    > 日期   : 2016-04-14
6  //*****
7  *
8  module adder_display(
9      //时钟与复位信号
10     input clk,
11     input resethn,    //后缀"n"代表低电平有效
12
13     //拨码开关，用于选择输入数和产生 cin
14     input input_sel, //0:输入为加数 1 (add_operand1); 1:为加数 2 (add_operand2)
15     input sw_cin,
16
17     //led 灯，用于显示 cout
18     output led_cout,
19
20     //触摸屏相关接口，不需要更改
21     output lcd_rst,
22     output lcd_cs,
23     output lcd_rs,
24     output lcd_wr,
25     output lcd_rd,
26     inout[15:0] lcd_data_io,
27     output lcd_bl_ctr,
28     inout  ct_int,
29     Inout  ct_sda,
30     output ct_scl,
```

```
30     output ct_rstn
31 );
32
33 //-----{调用加法模块}begin
34     reg [31:0] adder_operand1;
35     reg [31:0] adder_operand2;
36     wire      adder_cin;
37     wire [31:0] adder_result;
38     wire      adder_cout;
39     adder adder_module(
40         .operand1(adder_operand1),
41         .operand2(adder_operand2),
42         .cin      (adder_cin      ),
43         .result   (adder_result   ),
44         .cout     (adder_cout     )
45     );
46     assign adder_cin = sw_cin;
47     assign led_cout  = adder_cout;
48 //-----{调用加法模块}end
49
50 //-----{调用触摸屏模块}begin-----//
51 //-----{实例化触摸屏}begin
52 //此小节不需要更改
53     reg      display_valid;
54     reg [39:0] display_name;
55     reg [31:0] display_value;
56     wire [5 :0] display_number;
57     wire      input_valid;
58     wire [31:0] input_value;
59
60     lcd_module lcd_module(
61         .clk          (clk          ),    //10Mhz
62         .resetn       (resetn       ),
63
64         //调用触摸屏的接口
65         .display_valid (display_valid ),
66         .display_name  (display_name  ),
67         .display_value (display_value ),
68         .display_number (display_number),
69         .input_valid   (input_valid   ),
70         .input_value   (input_value   ),
71
72         //lcd 触摸屏相关接口，不需要更改
73         .lcd_rst       (lcd_rst       ),
74         .lcd_cs        (lcd_cs        ),
```

```
75         .lcd_rs      (lcd_rs      ),
76         .lcd_wr      (lcd_wr      ),
77         .lcd_rd      (lcd_rd      ),
78         .lcd_data_io  (lcd_data_io ),
79         .lcd_bl_ctr   (lcd_bl_ctr  ),
80         .ct_int       (ct_int      ),
81         .ct_sda       (ct_sda      ),
82         .ct_scl       (ct_scl      ),
83         .ct_rstn      (ct_rstn     )
84     );
85 //-----{实例化触摸屏}end
86
87 //-----{从触摸屏获取输入}begin
88 //根据实际需要输入的数修改此小节,
89 //建议对每一个数的输入, 编写单独一个 always 块
90     //当 input_sel 为 0 时, 表示输入数为加数 1, 即 operand1
91     always @(posedge clk)
92     begin
93         if (!resetn)
94         begin
95             adder_operand1 <= 32'd0;
96         end
97         else if (input_valid && !input_sel)
98         begin
99             adder_operand1 <= input_value;
100         end
101     end
102
103     //当 input_sel 为 1 时, 表示输入数为加数 2, 即 operand2
104     always @(posedge clk)
105     begin
106         if (!resetn)
107         begin
108             adder_operand2 <= 32'd0;
109         end
110         else if (input_valid && input_sel)
111         begin
112             adder_operand2 <= input_value;
113         end
114     end
115 //-----{从触摸屏获取输入}end
116
117 //-----{输出到触摸屏显示}begin
118 //根据需要显示的数修改此小节,
119 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
```

```
120 //44 块显示区域从 1 开始编号，编号为 1~44，
121     always @(posedge clk)
122     begin
123         case(display_number)
124             6'd1 :
125                 begin
126                     display_valid <= 1'b1;
127                     display_name  <= "ADD_1";
128                     display_value <= adder_operand1;
129                 end
130             6'd2 :
131                 begin
132                     display_valid <= 1'b1;
133                     display_name  <= "ADD_2";
134                     display_value <= adder_operand2;
135                 end
136             6'd3 :
137                 begin
138                     display_valid <= 1'b1;
139                     display_name  <= "RESUL";
140                     display_value <= adder_result;
141                 end
142             default :
143                 begin
144                     display_valid <= 1'b0;
145                     display_name  <= 40'd0;
146                     display_value <= 32'd0;
147                 end
148             endcase
149         end
150 //-----{输出到触摸屏显示}end
151 //-----{调用触摸屏模块}end-----//
152 endmodule
```

6 定点加法实验拓展

根据之前所述，可以画出定点加法实验的顶层模块框图，如下图：

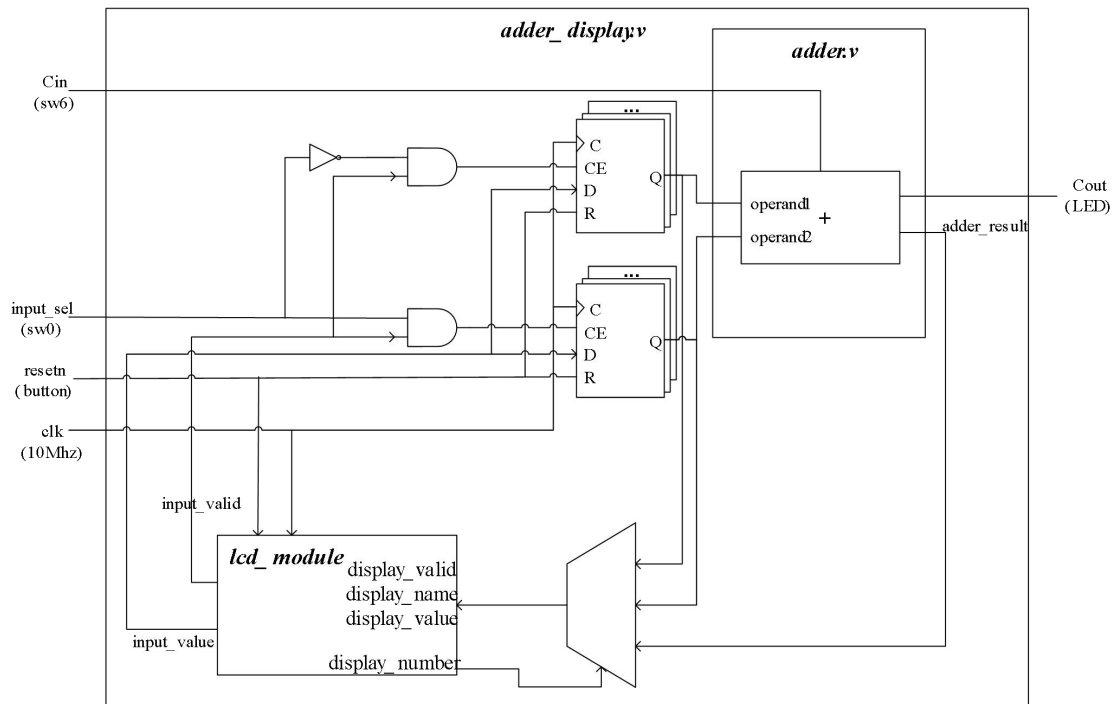


图 2.30 定点加法参考设计的顶层模块框图

从图中可以看到，外围模块 `adder_display` 内部调用了 `adder` 和 `lcd_module` 模块。在外部除了时钟和复位信号外，还有 `cin` 和 `input_sel` 通过拨码开关输入，以及 `cout` 输出到 led 灯上。

从实验二开始，针对每个实验需要画出类似图 2.40 的顶层模块。并且对内部功能模块(此处为 `adder.v`)需要画出原理图或实现框图，此处由于 `adder.v` 就用“+”实现，比较简单，故未给出加法模块的原理图。

本例中提供的实验参考设计主用于熟悉软硬件平台的，加法也是直接使用加号实现的。

学生可以在走通软硬件流程后，建议尝试修改加法的实现，可以使用 1 位全加器搭建出 32 位的加法，搭建的方法就有很多了，可以根据已学定点加法实现方法选择性地搭建串行进位加法器或先行进位加法器等，此时需要画出对应的原理图。